

6 Sciospec Communication Interface

The Sciospec Communication Interface (COMinterface) enables the user to access all functions of the device by using any of the available master interfaces. The actual command structure is identical in all connection types.

6.1 Syntax

The general structure of each communication with a Sciospec device:

- The communication is done by frames
- Each communication frame is constructed as follows
 - 1 byte command-Tag (Frame-Start)
 - 1 byte number of data-bytes (0...255)
 - 0...255 data-bytes
 - 1 byte Command-Tag (Frame-End)
- The command-tag identifies the command (see Command list)
- Frame-Start and -End must be identical

Example: "System-Ready-Message"

CMD-Tag	Number of bytes	Data	CMD-Tag
0x18	0x01	0x84	0x18

6.2 Acknowledge messages

- Communication-frames with incorrect syntax will cause a "Frame-Not-Acknowledge" message
- If the transmission of a communication-frame is interrupted for more than 10 ms a "Timeout" message is send
- Every invalid command-tag will cause a "Not-Acknowledge" message
- Every valid command is acknowledged with an acknowledge command [ACK]
- For commands with a return value the returning frame comes before the acknowledge message
- When commands are sent during the current measurements, measurement data can be transmitted between the command and the following returning frame and the acknowledge-message (commands are handled asynchronously)
- Before sending a new command, the resulting acknowledge or not acknowledge of the previous command has to be awaited.

The ACK-Frame:

0x18	0x01	[ACK]	0x18
General System Messages			
0x01	Frame-Not-Acknowledge: Incorrect syntax		

General System Messages	
0x02	Timeout: Communication-timeout (less data than expected)
0x04	Wake-Up Message: System boot ready
0x11	TCP-Socket: Valid TCP client-socket connection
0x81	Not-Acknowledge: Command has not been executed
0x82	Not-Acknowledge: Command could not be recognized
0x83	Command-Acknowledge: Command has been executed successfully
0x84	System-Ready Message: System is operational and ready to receive data

Device Specific System Messages	
0x90	Overcurrent Detected Value of DC current on W-ports exceeds capability of configured current range
0x91	Overvoltage Detected Value of DC voltage difference between R and WS port exceeds capability of configured voltage range

❗ For use of the impedance measurement module within a ISX-5/MSX-8 platform system refer to the respective system manual for guidance on specific communication within those systems. Commands and functionality are the same as described here, but additional framing and setup commands will be required due to the multi slot system structure within ISX-5/MSX-8.

6.3 Abbreviations

abbreviation	full name
[CT]	command tag
[LE]	length
[OB]	option byte
[CD]	command data

6.4 Command list

The leading hex code of each command heading represents the [command code] of the respective function.

0x90 - Save Settings

0x97 - Set Options
0x98 - Get Options
0xA1 - Reset System
0xB0 - Set FE Settings
0xB1 - Get FE Settings
0xB2 - Set ExtensionPort Channel
0xB3 - Get ExtensionPort Channel
0xB5 - Get ExtensionPort Module
0xB6 - Set Setup
0xB7 - Get Setup
0xB8 - Start Measure
0xB9 - Set Sync Time
0xBA - Get Sync Time
0xBD - Set Ethernet Configuration
0xBE - Get Ethernet Configuration
0xCF - TCP connection watchdog
0xD0 - Get ARM firmware ID
0xD1 - Get Device ID
0xD2 - Get FPGA firmware ID
0xB3 - Get ExtensionPort Channel

6.5 Command description

6.5.1 0x90 - Save Settings

General Syntax

[CT] 00 [CT]

Return

ACK

Description

Saves the following parameters permanently into the flash memory of the ISX-3:

- NTC Parameters 1 and 2
- Parameter stack synchronization time

This command can only be used if no measurement is currently running.

6.5.2 0x97 - Set Options

General Syntax

[CT] [LE] [OB] [CD] [CT]

Return

ACK

[OB]

Function	code
Activate time stamp in ms	0x01
Activate time stamp in μ s	0x02
Activate current range	0x04

Active time stamp in ms

Configuration of the Instrument. Activate time stamp in ms of measured data. Time stamp added to measurement data in 4Byte unsigned integer. If measured time exceed 0xFFFFFFFF (about 1193h) time will start over at 0x00000000.

Syntax

- Syntax set: [CT] 02 01 [CD] [CT]

[CD]

- 0x01 Enable Time Stamp
- 0x00 Disable Time Stamp

Depending on this setting the return frame of the measured data changes (see command B8)

Remarks:

It is not possible to change this setting while a measurement is running. This setting cannot be saved persistently. It is not possible to activate the time stamp in μ s and in ms. Only the last configuration will be valid.

Active time stamp in μ s

Configuration of the Instrument. Activate time stamp in μ s of measured data. Time stamp added to measurement data in 5Byte unsigned integer. If measured time exceed 0xFFFFFFFF (about 305,4h) time will start over at 0x0000000000.

Syntax

- Syntax set: [CT] 02 02 [CD] [CT]

[CD]

- 0x01 Enable Time Stamp
- 0x00 Disable Time Stamp

Depending on this setting the return frame of the measured data changes (see command B8)

Remarks:

It is not possible to change this setting while a measurement is running. This setting cannot be saved persistently. It is not possible to activate the time stamp in μ s and in ms. Only the last configuration will be valid.

Activate current range

Configuration of the Instrument. Activate current range of measured data. The current range is added to the measurement data as 1Byte unsigned integer. A description of possible values can be found in "0xB0 - Set FE Settings".

Syntax

- Syntax set: [CT] 02 04 [CD] [CT]

[CD]

- 0x01 Enable current range
- 0x00 Disable current range

Depending on this setting the return frame of the measured data changes (see command B8)

Remarks:

It is not possible to change this setting while a measurement is running. This setting cannot be saved persistently.

6.5.3 0x98 - Get Options

General Syntax

[CT] [LE] [OB] [CT]

Return

[CT] [LE] [OB] [CD] [CT]

ACK

[OB]

Function	code
Time stamp	0x01 0x02
frequency range	0x03
current range	0x04

Time stamp

Returns the currently configured options of the instruments.

Syntax

- Syntax get: [CT] 01 [OB] [CT]
 - Return: [CT] 02 [OB] [CD] [CT]

[CD]

- Currently configured time stamp option
 - CD = 0x00: time stamp disabled
 - CD = 0x01: time stamp in ms
 - CD = 0x02: time stamp in μ s

frequency range

Returns available configurable frequency range of the instruments.

Syntax

- Syntax get: [CT] 01 03 [CT]

- Return: [CT] 02 03 [MinF] [MaxF] [CT]

[MinF]

- Minimum configurable frequency
- 4Byte Float in Hz

[MaxF]

- Maximum configurable frequency
- 4Byte Float in Hz

current range

Returns the currently configured options of the instruments.

Syntax

- Syntax get: [CT] 01 03 [CT]
 - Return: [CT] 02 03 [CD] [CT]

[CD]

- Currently configured current range option
CD = 0x00: current range disabled
CD = 0x01: current in ms

6.5.4 0xA1 - Reset System

complete restart of the system

General Syntax

[CT] 00 [CT]

Return

ACK
Wake-Up Message
System-Ready-Message

6.5.5 0xB0 - Set FE Settings

Frontend configuration

❗ The device has a stack length of 1 (ISX-3) or 2 (ISX-3 mini or ISX-3 with second channel option or InternalMux). So only 1 (or 2) FE settings can be stored. Sending the set FE settings command more than 1 (or 2) times results in a NACK return. Send following command to empty the stack:
B0 03 FF FF FF B0

General Syntax

[CT] 03 [measurement mode] [measurement channel] [current range settings] [CT]

[CT] 04 [measurement mode] [measurement channel] [current range settings] [voltage range settings] [CT]

The parameter of **[voltage range setting]** is optional and does not need to be sent. If **[voltage range setting]** is not sent the default value is used.

Return

ACK

[measurement mode]

Function	code
4 point configuration	0x02
3 point configuration	0x03
2 point configuration	0x01

[measurement channel]

Function	code
BNC Port (ISX-3mini: Port 1)	0x01
ExtensionPort	0x02
ExtensionPort2 (ISX-3mini: Port 2, ISX-3: optional, <i>InternalMux</i>)	0x03

[current range settings]

current measurement range	old range nomenclature	code
autoranging		0x00
± 10 mA	100	0x01
± 100 µA	10k	0x02
± 1 µA	1M	0x04
± 10 nA	100M	0x06

[voltage range settings]

voltage measurement range	code
autoranging	0x00
±1 V (default)	0x01
± 0.09 V	0x02

6.5.6 0xB1 - Get FE Settings

Returns the currently selected frontend configuration

General Syntax

[CT] 00 [CT]

Return

[CT] 04 [measurement mode] [measurement channel] [current range settings] [voltage range settings] [CT]

ACK

[measurement mode]

Function	code
4 point configuration	0x02
3 point configuration	0x03
2 point configuration	0x01

[measurement channel]

Function	code
BNC Port (ISX-3mini: Port 1)	0x01
ExtensionPort	0x02
ExtensionPort2 (ISX-3mini: Port 2, ISX-3: optional, <i>InternalMux</i>)	0x03

[current range settings]

current measurement range	formally	code
autoranging		0x00
± 10 mA	100	0x01
± 100 μ A	10k	0x02
± 1 μ A	1M	0x04
± 10 nA	100M	0x06

[voltage range settings]

voltage measurement range	code
autoranging	0x00
$\pm 1 \text{ V}$	0x01
$\pm 0.09 \text{ V}$	0x02

6.5.7 0xB2 - Set ExtensionPort Channel

Set the ExtensionPort channel settings.

General syntax

[CT] 04 [CP] [RP] [WS] [WP] [CT]

Return

ACK

ExtensionPort- channel settings

[CP]	(C-Port): Counter - Port Selection
[RP]	(R-Port): Reference - Port Selection
[WS]	(WS-Port): Working Sense - Port Selection
[WP]	(W-Port): Work - Port Selection

See documentation of the connected extension module for detailed information.

6.5.8 0xB3 - Get ExtensionPort Channel

Read the currently set ExtensionPort configuration.

General syntax

[CT] 00 [CT]

Return

[CT] 04 [CP] [RP] [WS] [WP] [CT]

ACK

ExtensionPort- channel settings

CP	(C-Port): Counter - Port Selection
RP	(R-Port): Reference - Port Selection
WS	(WS-Port): Working Sense - Port Selection

WP	(W-Port): Work - Port Selection
----	---------------------------------

See documentation of the connected extension module for detailed information.

6.5.9 0xB5 - Get ExtensionPort Module

Read the type of the currently connected extension module.

General syntax

[CT] 00 [CT]

Return

Optional channelCount is only valid in case of module = Mux32any2any2202

[CT] [LE] [ExtensionModule] [optional ChannelCount Ext*] [InternalModule] [optional ChannelCount Int*]
[CT]

ACK

[LE]

- 2 byte, unsigned integer
- 4 byte, unsigned integer if ExtensionModule = Mux32any2any2202
- 4 byte, unsigned integer if InternalModule = Mux32any2any2202
- 6 byte, unsigned integer if ExtensionModule = Mux32any2any2202 and InternalModule = Mux32any2any2202

[ExtensionModule]

This code represents the connected extension module.

Function	code
no module connected	0x00
MEArack	0x01
MuxModule32	0x02
ECIS Adapter	0x03
ExtensionPortAdapter	0x05
SlideChipAdapter	0x06
Mux32any2any (external)	0x07
DaQEisMux	0x08

Function	code
Mux32any2any2202 (external)	0x09

[InternalModule]

This code represents the internal module.

Function	code
no module connected	0x00
MuxModule16x4	0x01
MuxModule32x2	0x02
Mux32any2any (internal)	0x07
Mux32any2any2202	0x09

[optional ChannelCount*]: 2byte unsigned integer

6.5.10 0xB6 - Set Setup

General syntax

[CT] [LE] [OB] [CD] [CT]

Return

ACK

[OP]

	Function	code
Initialization and Configuration Commands	Init	0x01
	Add single frequency point	0x02
	Add frequency list	0x03
	Set amplitude	0x05
Compensation Commands	Start compensation	0x10
	Compensation acknowledge	0x11
	Compensation-interaction-request Load value	0x16

	Set compensation data	0x17
Loading Setup from Slot	Loading setup from slot	0x20
DC Bias	DC Bias	0x30
	DC Bias not-acknowledge	0x32
	DC Bias set value	0x33

Init

This option resets the currently configured setup and an empty setup is initialized.

Syntax:

- Syntax set: **[CT] 01 01 [CT]**

Add single frequency point

This command is used to add a single frequency point to the currently configured setup.

Syntax

- Syntax set: **[CT] [LE] 02 [CD] [CT]**

[CD]

- [LE] = 13 (frequency, precision, amplitude)

frequency			precision			amplitude		
MSB	...	LSB	MSB	...	LSB	MSB	...	LSB

In this case the point delay (= 0 ms) and phase sync is not used.

- [LE] = 13 + extended options length

frequency			precision			amplitude			point delay				use phase sync				excitation type			
MS B	...	LS B	MS B	...	LS B	MS B	...	LS B	EOP =01	MS B	...	LS B	EO P =02	MS B	...	LS B	EO P =03	MS B	...	LS B
obligatory									optional				optional				optional			

Frequency: frequency in Hz (4Byte float)

i Frequency must be > minimum frequency for measuring AC impedance or frequency must be equal to zero for measuring DC resistance.

Precision: precision value (4Byte float)

Amplitude:

- for voltage excitation: amplitude in V (peek value, 4Byte float)
- for current excitation: amplitude in A (peek value, 4Byte float)
- values outside of the following amplitude ranges will be acknowledged and corrected internally to the corresponding minimum or maximum
 - voltage excitation range: 0.1 mV to 1 V
 - current excitation amplitude range: 1 µA to 10 mA
 - please note that the specified excitation ranges are smaller then ranges listed above. Sciospec will only guarantee the correctness of the specified excitation ranges stated in the Technical Specifications.

Extended Options EOP:

- For each optional additional setting an EOP identifier needs to be send before the actual setting. Therefore any of the following options or multiple can be send.
- Point delay (EOP=01): delay between this frequency and the next frequency in µs (4Byte unsigned integer)
- Use phase sync (EOP=02): phase synchronous switch between this and the next frequency
 - Use: 0x00000001
 - Don't use: 0x00000000
- excitation type (EOP=03): Defines weather the excitation amplitude is in Volt or in Ampere
 - voltage excitation: 0x00000001 (default)
 - current excitation: 0x00000002

Examples

- B6 0D 02 46 FA 00 00 3F 80 00 00 3E 80 00 00 B6
 - Frequency = 32 kHz
 - precision = 1.0
 - amplitude = 0.25 V
 - NO extended options have been used
- B6 12 02 46 FA 00 00 3F 80 00 00 3E 80 00 00 02 00 00 00 01 B6
 - Frequency = 32 kHz
 - precision = 1.0
 - amplitude = 0.25 V
 - phase synchronization = used
 - One extended option has been used

Add frequency list

This command is used to add multiple frequencies to the currently configured setup.

Syntax

- Syntax set: [CT] [LE] 03 [CD] [CT]

[CD]

- [LE] = 16

start-frequency	stop-frequency	count	scale	precision	amplitude
-----------------	----------------	-------	-------	-----------	-----------

MSB	...	LSB	MSB	...	LSB	MSB	...	LSB	MSB	MSB	...	LSB	MSB	...	LSB
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

In this case the point delay (= 0 ms) and phase sync is not used.

- [LE] = 16 + extended options length

start-frequency			stop-frequency			count			scale	precision			amplitude			point delay				use phase sync				excitation type			
M	.	L	M	.	L	M	.	L	M	M	.	L	M	.	L	E	M	.	L	E	M	.	L	E	M	.	L
S	.	S	S	.	S	S	.	S	S	S	.	S	S	.	S	O	S	.	S	O	S	.	S	O	S	.	S
B	.	B	B	.	B	B	.	B	B	B	.	B	B	.	B	P	B	.	B	P	B	.	B	P	B	.	B
																=				=				=			
																0				0				0			
																1				2				3			
obligatory																optional				optional				optional			

- Start-Frequency: start-frequency of the frequency block in Hz (4Byte float)
- Stop-Frequency: stop-frequency of the frequency block in Hz (4Byte float)
- Count: number of frequency steps used (4Byte float, rounded to the next smaller integer)
- Scale: linear or logarithmic scale (1Byte integer)
 - linear: 0
 - logarithmic: 1
- Precision: precision value (4Byte float)
- Amplitude:
 - for voltage excitation: amplitude in V (peek value, 4Byte float)
 - for current excitation: amplitude in A (peek value, 4Byte float)
 - values outside of the following amplitude ranges will be acknowledged and corrected internally to the corresponding minimum or maximum
 - voltage excitation range: 0.1 mV to 1 V
 - current excitation amplitude range: 1 µA to 10 mA
 - please note that the specified excitation ranges are smaller than ranges listed above. Sciospec will only guarantee the correctness of the specified excitation ranges stated in the Technical Specifications.

Extended Options EOP:

- For each optional additional setting an EOP identifier needs to be send before the actual setting. Therefore any of the following options or multiple can be send.
- Point delay: delay between this frequency and the next frequency in µs (4Byte unsigned integer)
- Use phase sync: phase synchronous switch between this and the next frequency
 - Use: 0x00000001
 - Don't use: 0x00000000
- excitation type (EOP=03): Defines weather the excitation amplitude is in Volt or in Ampere
 - voltage excitation: 0x00000001 (default)
 - current excitation: 0x00000002

Example

- B6 20 03 44 7A 00 00 4B 18 96 80 41 20 00 00 01 3F 80 00 00 3E 80 00 00 01 00 00 03 E8 02 00 00 00 00 B6
- start frequency = 1 kHz
- stop frequency = 10 MHz
- count = 10
- scale = logarithmic
- precision = 1.0
- amplitude = 0.25 V
- point delay = 1000 μ s
- phase sync = disable

Set amplitude

This command sets the amplitude.

Syntax

Two options are available:

- Syntax set all amplitudes with one command: [CT] 06 05 [ExcitationType] [Amplitude] [CT]
- Syntax set amplitudes row-wise: [CT] 08 05 [Row] [ExcitationType] [Amplitude] [CT]

[ExcitationType]

- Length: 1 byte
- 0x01 - voltage
- 0x02 - current

[Amplitude]

- Length: 4 byte
- Data format: float
- for voltage excitation: amplitude in V (peek value, 4Byte float)
- for current excitation: amplitude in A (peek value, 4Byte float)
- values outside of the following amplitude ranges will be acknowledged and corrected internally to the corresponding minimum or maximum
 - voltage excitation range: 0.1 mV to 1 V
 - current excitation amplitude range: 1 μ A to 10 mA
 - please note that the specified excitation ranges are smaller than ranges listed above. Sciospec will only guarantee the correctness of the specified excitation ranges stated in the Technical Specifications.

[Row]

- Length: 2 byte
- $0 \leq \text{row} \leq \text{maximum row count}$

start compensation

A detailed example of the required communication necessary to compensate a setup can be found in [Example of impedance compensation procedure](#).

This command starts the compensation of the setup.

Syntax

- Syntax set: **[CT] 01 10 [CT]**

compensation acknowledge

The compensation acknowledge (CACK) is used to answer a "Compensation-Interaction-Request"

Syntax

- Syntax: **[CT] 01 11 [CT]**

Compensation not-acknowledge

The compensation acknowledge (CNACK) is used to answer a "Compensation-Interaction-Request"

Syntax

- Syntax: **[CT] 01 12 [CT]**

compensation-interaction-request Open

This is a message from the ISX-3 to request an open configuration on the measurement channel. This request must be answered with a CACK to continue the compensation or with a CNACK to abort the compensation.

compensation-interaction-request Short

This is a message from the ISX-3 to request a short configuration on the measurement channel. This request must be answered with a CACK to continue the compensation or with a CNACK to abort the compensation.

compensation-interaction-request Load

This is a message from the ISX-3 to request a load configuration on the measurement channel. This request must be answered with a CACK to continue the compensation or with a CNACK to abort the compensation.

Before the answer to this request is send the load value and type must have been send to the device (Option 0x16).

compensation-interaction-request Load value

The connected load value must have been send in the command data part [CD] of the command.

Syntax

- Syntax set: **[CT] 06 16 [LoadType] [Value] [CT]**

[LoadType]

- Length: 1 byte
- 0x01 - resistance
- 0x02 - capacitor

[Value]

- Length: 4 byte
- Data format: float
- Unit: Ohm for [LoadType] = resistance, Farad for [LoadType] = capacitor

Example

- **[CT] 06 16 01 44 7A 00 00 [CT]** → **resistance = 1000 Ohm**

set compensation data

This command configures the compensation data of the configured setup.

Syntax

- Syntax reset compensation data: **[CT] 02 17 01 [CT]**
- Syntax set data: **[CT] [LE] 17 02 [Channel] [Row] [Open Re] [Open Im] [Short Re] [Short Im] [Load Re] [Load Im] [CT]**

[Channel]	[Row]	[Open Re]	[Open Im]	[Short Re]	[Short Im]	[Load Re]	[Load Im]
1 byte	2 byte	4 byte float	4 byte float	4 byte float	4 byte float	4 byte float	4 byte float

Loading setup from slot

It is possible to load (recalling) all information required for a measurement from the internal storage of the impedance analyzer. The system holds up to 255 setup configurations including any compensation data. For addressing, the configurations are numerated from slot 1 to slot 255.

Syntax

- Syntax loading from slot: **[CT] 02 20 [Slot] [CT]**

[Slot]

- Length: 1 byte

Example

- **[CT] 02 20 05 [CT]** → [load configuration from slot 5](#)

DC Bias

This command activates or deactivates the DC Bias on the configured channel. When the DC Bias is activated DC Bias is set to 0V and then regulated towards the selected value. This process can be aborted by sending the “DC Bias-Not-Acknowledge”.

If the DC Bias is reached a “DC Bias-Acknowledge” is send from the ISX-3 / ISX-3 mini.

If the DC Bias could not reached “DC Bias-Not-Acknowledge” is send from the ISX-3 / ISX-3 mini.

Syntax

- Syntax set: **[CT] 02 30 [CD] [CT]**

[CD]

- Length: 1 byte
- 0x00 - deactivate
- 0x01 - activate

DC Bias not-acknowledge

This command is to abort the DC Bias regulation.

Syntax

- Syntax: **[CT] 01 32 [CT]**

DC Bias set value

This command sets the value for the DC Bias.

Syntax

- Syntax set: **[CT] 05 33 [bias value] [CT]**

[bias value]

- Length: 4 byte
- Data format: float
- Unit: V (volt)
- range: + 1 V to - 1 V

Example

- [CT] 05 33 **3F 80 00 00** [CT] → [Sets the bias value to 1 V.](#)

- ① How to regulated the DC Bias after the measuring section or the measuring object changed:
- send the **DC Bias set value** command (Syntax set: [CT] 05 33 [bias value] [CT]) → not necessary if the set value doesn't change
 - send the **DC Bias** command (Syntax: [CT] 02 30 [CD] [CT])

6.5.11 0xB7 - Get Setup

General syntax

[CT] [LE] [OB] [CD] [CT]

Return

[CT] [LE] [OB] [CD] [CT]

ACK

[OP]

Function	code
Get total number of frequencies	0x01
Get information of frequency point	0x02
Get frequency list	0x04
compensation-interaction-request Open	0x13
compensation-interaction-request Short	0x14
compensation-interaction-request Load	0x15
save setup to slot	0x20

Function	code
Get DC Bias	0x33

Get total number of frequencies

This command reads the total number of frequencies configured in the setup.

Syntax

- Syntax get: **[CT] 01 01 [CT]**
 - Return: **[CT] 03 01 [CD] [CT]**

[CD]

- number of rows configured
- Length: 2 byte
- Data format: unsigned integer

Get information of frequency point

This command gets information of a configured point of the setup.

Syntax

- Syntax get: **[CT] XX 02 [CD] [EOPs] [CT]**
 - Return: **[CT] XX 02 [Frequency] [Precision] [Signal amplitude] [EOP] [EOP-value] [CT]**

[CD]

- row number
- Length: 2 byte
- Data format: unsigned integer

[Frequency]

- Length: 4 byte
- Data format: float
- Unit: Hz

[Precision]

- Length: 4 byte
- Data format: float

[Signal amplitude]

- Length: 4 byte
- Data format: float
- voltage amplitude in V
- current amplitude in A

[EOPs]

- optional list of extended options
- each extended option can be on of the following

EOP	
0x01	point delay
0x02	phase sync
0x03	excitation type

Get frequency list

This command gets a list of frequencies configured in this setup.

Syntax

- Syntax get: **[CT] 01 04 [CT]**
 - Return: **[CT] [LE] 04 [4 Byte float frequency₁] [...] [4 Byte float frequency_N] [CT]**

[4 Byte float frequency₁] [...] [4 Byte float frequency_N]

- *In the case of "get frequency list" cmd one data frame is limited to a total of 252 bytes of data. The returning command will be split into multiple separate frames, if this number is exceeded. For example if the setup contains 64 frequency points the first 62 will be transmitted in a frame containing 249 bytes ($=62*4+1$) and in a separate frame containing 9 bytes ($=2*4+1$) bytes of data.*

compensation-interaction-request Open

This is a message send by the ISX-3 to request an open configuration on the measurement channel. This request must be answered with a CACK to continue the compensation or with a CNACK to abort the compensation.

compensation-interaction-request Short

This is a message send by the ISX-3 to request a short configuration on the measurement channel. This request must be answered with a CACK to continue the compensation or with a CNACK to abort the compensation.

compensation-interaction-request Load

This is a message send by the ISX-3 to request a load configuration on the measurement channel. This request must be answered with a CACK to continue the compensation or with a CNACK to abort the compensation.

Before the answer to this request is send the load value and type must have been send to the device (Option 0x16).

save setup to slot

This command saves the setup-configuration.

Syntax

- Syntax: **[CT] 02 20 [Slot] [CT]**

[Slot]

- Slot number
- Length: 1 byte

Get DC Bias

This command reads the currently configured DC Bias from the device.

Syntax

- Syntax get: [CT] 01 33 [CT]
 - Return: [CT] 05 33 [DC Bias] [CT]

[DC Bias]

- Length: 4 byte
- Data format: float
- Unit: V

6.5.12 0xB8 - Start Measure

Starts the measurement.

General syntax

[CT] [LE] [OB] [CD] [CT]

Return

If time stamp and current range are disabled (see command 0x97 and 0x98)

[CT] 0A [ID] [Real part] [Imaginary part] [CT]

Else if time stamp in ms is enabled (see command 0x97 and 0x98)

[CT] 0E [ID] [Time stamp] [Real part] [Imaginary part] [CT]

Else if time stamp in μ s is enabled (see command 0x97 and 0x98)

[CT] 0F [ID] [Time stamp] [Real part] [Imaginary part] [CT]

Else if current range is enabled (see command 0x97 and 0x98)

[CT] 0B [ID] [Current Range] [Real part] [Imaginary part] [CT]

Else if time stamp and current range are enabled (see command 0x97 and 0x98)

[CT] 0F [ID] [Time stamp] [Current Range] [Real part] [Imaginary part] [CT]

[ID]

- ID number of the frequency point
- Length: 2 byte
- Data format: unsigned integer

[Time stamp]

- time stamp of the frequency point
- Length: 4 byte or 5 byte (depending on configured time stamp)
- Data format: unsigned integer
- Unit: ms or μ s (depending on configured time stamp)

[Current Range]

- the current range of that frequency point; in case of auto ranging this could change from point to point
- Length 1 byte
- Data format: unsigned integer

[Real part]

- real part of impedance
- Length: 4 byte
- Data format: float

[Imaginary part]

- imaginary part of impedance
- Length: 4 byte
- Data format: float

 Please refer to the Appendix for a description of the data format.

 In case of a DC measurement (excitation frequency = 0Hz), the real part contains the resistance in Ohm and the Imaginary part equals zero.

[OP]

Function	code
Stop measurement	0x00
Start measurement	0x01
Start EIT measurement	0x02

Stop measurement

Stops the measurement.

Syntax

- Syntax: **[CT] 01 00 [CT]**

Start measurement

Starts the measurement.

Syntax

- Syntax: **[CT] 03 01 [CD] [CT]**

[CD]

- number of spectra to be measured
- Length: 2byte
- Data format: integer value
- The setting 0 starts a continuous measurement. Send the command (B8 01 00 B8) to stop the continuous run.

- example: B8 03 01 00 01 B8 - to start a measurement and stop it automatically after one measurement spectra per channel configuration.

❗ If the measurement has been started with a repeat greater than 0 no stop command is required. The data will be transmitted as soon as each frequency point has been measured. The system therefore **does not** wait for the spectrum to be completed before it will send the data.

6.5.13 0xB9 - Set Sync Time

Set the synchronization time in μs . (Time between the measurement of two spectra)

❗ This is also the time the SyncOut signal is in its low state between two measurements. Use save settings command (0x90) to save this parameter persistent.

General syntax

[CT] [LE] [Sync time] [CT]

Return

ACK

[Sync time]

- Length: 4 byte
- Data format: integer value
- Unit: μs
- Default: 0 μs
- Min: 0 μs
- Max 180s = 180E6 μs (0x0ABA9500)

6.5.14 0xBA - Get Sync Time

Reads the currently configured synchronization time.

General syntax

[CT] 00 [CT]

Return

[CT] 04 [Sync time] [CT]

ACK

[Sync time]

- Length: 4 byte
- Data format: integer value

- Unit: μ s

6.5.15 0xBD - Set Ethernet Configuration

Configure DHCP setting and IP address.

General Syntax

[CT] [LE] [OB] [CD] [CT]

Return

ACK

[LE]

- Represents the byte count of the command frame and varies with the amount [data] bytes. [length] is always the amount of [data]-bytes + 1.

[CD]

- Command data whose syntax and content depends on the used [option byte].

[OB]

Function	code
IP address	0x01
DHCP on/of	0x03

IP address

- Set static IPv4 address of the device.
- This address will only be used, when DHCP is disabled.
- The default setting is 0.0.0.0
- Syntax: [CT] [05] [01] [address] [CT]
- [address]
 - Desired address.
 - Length: 4 byte
 - Data format: byte-wise unsigned integer
 - Value range of each byte: 0 ... 255

DHCP on/off

- Activate/deactivate DHCP usage.
-  To apply this setting, a system reboot is required.
- Syntax: [CT] [02] [03] [switch] [CT]
- [switch]
 - Length: 1 byte
 - Values:
 - 0x00 - off
 - 0x01 - on (default)

❗ To save these settings persistently, the "save settings" command must be sent before powering off the device.

6.5.16 0xBE - Get Ethernet Configuration

Read out parameters like DHCP setting, MAC and IP address.

General Syntax

[CT] [LE] [OB] [CT]

Return

[CT] [LE] [OB] [CD] [CT]

ACK

[LE]

- Represents the byte count of the command frame and varies with the amount [data] bytes. [length] is always the amount of [data]-bytes + 1.

[CD]

- Command data whose syntax and content depends on the used [option byte].

[OB]

Function	get
IP address	0x01
MAC address	0x02
DHCP on/off	0x03

IP address

- Read out currently allocated IPv4 address of the device. This is the actual address the device
- Get syntax: [CT] [01] [01] [CT]
- Return syntax: [CT] [05] [01] [address] [CT]
- [address]
 - Current address.
 - Length: 4 byte
 - Data format: byte-wise unsigned integer

MAC address

- Read out currently allocated MAC address of the device. This is the actual address the device
- Get syntax: [CT] [01] [02] [CT]

- Return syntax: [CT] [07] [02] [address] [CT]
- [address]
 - MAC address.
 - Length: 6 byte
 - Data format: byte-wise unsigned integer

DHCP on/off

- Read out the current DHCP setting.
- Get syntax: [CT] [01] [03] [CT]
- Return Syntax: [CT] [02] [03] [switch] [CT]
- [switch]
 - Length: 1 byte
 - Values:
 - 0x00 - off
 - 0x01 - on

6.5.17 0xCF - TCP connection watchdog

General Syntax

[CT] 05 00 [interval] [CT]

Return

ACK

Description

The device checks periodically if the connection from the TCP socket to the client is still active. This is done by the device by periodically sending the TCP socket acknowledge command (see chapter "Acknowledge messages"). If the command cannot be send, i.e. because of a lost client connection, the socket server state will be reset to "listening". Thus, a new socket-client connection can be established.

This command provides access to setting the interval for sending the TCP socket acknowledge command.

[interval]

- Set the interval for TCP communication watchdog
- Length: 4 Byte
- Data format: unsigned integer
- Values:
 - Minimum: 1 s
 - Maximum: 600 s
 - Increment: 1 s
 - Default: 60 s

 To save these settings persistently, the "save settings" command must be sent before powering off the device.

6.5.18 0xD0 - Get ARM firmware ID

Syntax

D0 00 D0

Return

D0 06 [developer information] [revision number] [build number] D0
ACK

Description

- Reads out version number of ARM firmware.

[developer information]

- This information is for internal development purposes only.
- Length: 2 Byte

[revision number]

- Length: 2 Byte
- Data format: unsigned integer

[build number]

- Length: 2 Byte
- Data format: unsigned integer

6.5.19 0xD1 - Get Device ID

Syntax

[CT] 00 [CT]

Return

[CT] [LE] [general information] [developer information] [CT]
ACK

Description

- Read out device information
- Includes all information necessary to identify a Sciospec device.

[general information]

This information can also be found on the serial number label on the case of the device.

Byte	Description	Comments
1	version of the general information part	- Defines the format of the following items. There is only version 1 so far. - data format: unsigned integer
2-3	device identifier	- Unique number, which identifies the device types. - data format: unsigned integer

4-5	serial number	- Unique identifying number for all Sciospec devices. - data format: unsigned integer
6-7	date of delivery	- Byte 6: Year since 2010 (0x00 = 2010, ..., 0xFF = 2265) - Byte 7: Month (0x01 = january, ..., 0x0C = december)

[developer information]

This information is for internal development purposes only.

6.5.20 0xD2 - Get FPGA firmware ID

Syntax

D2 00 D2

Return

D2 09 [developer information] [revision number] [build number] D2
ACK

Description

- Reads out version number of ARM firmware.

[developer information]

- This information is for internal development purposes only.
- Length: 5 Byte

[revision number]

- Length: 2 Byte
- Data format: unsigned integer

[build number]

- Length: 2 Byte
- Data format: unsigned integer

6.5.21 Example of impedance compensation procedure

This example demonstrates the communication required to compensate an already configured setup.

Direction of communication:

PC to ISX-3 / ISX-3mini							
ISX-3 / ISX-3mini to PC							
↓	#	Command	Communication				
	1	Initiating the compensation routine	<table border="1"> <tr> <td>B6</td><td>01</td><td>10</td><td>B6</td></tr> </table>	B6	01	10	B6
B6	01	10	B6				
	2	Acknowledge	<table border="1"> <tr> <td>18</td><td>01</td><td>83</td><td>18</td></tr> </table>	18	01	83	18
18	01	83	18				

	3	Request to leave the terminals open	<table><tr><td>B7</td><td colspan="2">01</td><td colspan="2">13</td><td colspan="2">B7</td></tr></table>								B7	01		13		B7		
	B7	01		13		B7												
	4	compensation acknowledge	<table><tr><td>B6</td><td colspan="2">01</td><td colspan="2">11</td><td colspan="2">B6</td></tr></table>								B6	01		11		B6		
	B6	01		11		B6												
	5	Acknowledge	<table><tr><td>18</td><td colspan="2">01</td><td colspan="2">83</td><td colspan="2">18</td></tr></table>								18	01		83		18		
	18	01		83		18												
	compensation measurement on open terminal is performed																	
	6	Request to short all terminals	<table><tr><td>B7</td><td colspan="2">01</td><td colspan="2">14</td><td colspan="2">B7</td></tr></table>								B7	01		14		B7		
	B7	01		14		B7												
	7	compensation acknowledge	<table><tr><td>B6</td><td colspan="2">01</td><td colspan="2">11</td><td colspan="2">B6</td></tr></table>								B6	01		11		B6		
	B6	01		11		B6												
	8	Acknowledge	<table><tr><td>18</td><td colspan="2">01</td><td colspan="2">83</td><td colspan="2">18</td></tr></table>								18	01		83		18		
	18	01		83		18												
	compensation measurement of shorted terminals is performed																	
	9	Request to connect a known load impedance to the device	<table><tr><td>B7</td><td colspan="2">01</td><td colspan="2">15</td><td colspan="2">B7</td></tr></table>								B7	01		15		B7		
B7	01		15		B7													
10	Set known value of the load impedance (e.g. 1000hms)	<table><tr><td>B6</td><td>06</td><td>16</td><td>01</td><td>42</td><td>C8</td><td>00</td><td>00</td><td>B6</td></tr></table>								B6	06	16	01	42	C8	00	00	B6
B6	06	16	01	42	C8	00	00	B6										
11	Acknowledge	<table><tr><td>18</td><td colspan="2">01</td><td colspan="2">83</td><td colspan="2">18</td></tr></table>								18	01		83		18			
18	01		83		18													
12	compensation acknowledge	<table><tr><td>B6</td><td colspan="2">01</td><td colspan="2">11</td><td colspan="2">B6</td></tr></table>								B6	01		11		B6			
B6	01		11		B6													
13	Acknowledge	<table><tr><td>18</td><td colspan="2">01</td><td colspan="2">83</td><td colspan="2">18</td></tr></table>								18	01		83		18			
18	01		83		18													
compensation measurement of load terminals is performed																		
14	compensation done	<table><tr><td>B7</td><td colspan="2">01</td><td colspan="2">10</td><td colspan="2">B7</td></tr></table>								B7	01		10		B7			
B7	01		10		B7													

6.5.22 Example of dc bias compensation procedure

This example demonstrates the communication required to compensation an already configured setup. It is essential the frontend settings (channel, measurement mode and range) are correctly initialized before starting the dc bias compensation routine

Direction of communication:

PC to ISX-3 / ISX-3mini

ISX-3 / ISX-3mini to PC			
↓	#	Command	Communication
	1	Setting the dc bias voltage	B6 05 33 3F 00 00 00 B6
	2	Acknowledge	18 01 83 18
	3	Init the dc bias compensation	B6 01 30 B6
	4	Acknowledge	18 01 83 18
	compensation measurement is performed		
	5	compensation DC Bias acknowledge	B7 01 31 B7
	6	Getting the dc bias voltage	B7 01 33 B7
	7	DC Bias	B7 05 33 3E FF FE B0 B7
	8	Acknowledge	18 01 83 B18

6.6 Code Example (ANSI C)

```
1  /*
2
3  =====
4  ==
5  Name      : ISX-3_C-Code.c
6  Author    : Sciospec
7  Version   : Revision 1
8  Copyright : Sciospec 2022
9  Description: ISX-3 Communication Demo in C, Ansi-style
10
11 =====
12 ==
13 */
14
15 #include <stdio.h>
16 #include <stdlib.h>
17
18 byte connectToDevice(HANDLE* handle);
```

```

16 byte readAck(HANDLE handle);
17 void readData(HANDLE handle, byte* buffer, int bytesToRead);
18 void writeToDevice(HANDLE handle, byte* data, DWORD dataCount);
19
20 int main(void) {
21
22     HANDLE handle;
23     if(!connectToDevice(&handle))
24         return -1;
25
26     //This will force stdout to be unbuffered.
27     setvbuf (stdout, NULL, _IONBF, 0);
28
29     printf("Connection established\n");
30     byte *cmd, *readBuffer;
31
32
33
34     /* ***** *
35     *           *
36     *   Initialize Setup   *
37     *           *
38     * ***** */
39     printf("Initialize Setup.\n");
40     int numberOfBytes = 4;
41     cmd = (byte*)malloc(sizeof(byte)*numberOfBytes);
42     cmd[0] = 0xB6; cmd[1] = 0x01; cmd[2] = 0x01; cmd[3] = 0xB6;
43     writeToDevice(handle, cmd, numberOfBytes);
44     free(cmd);
45     readAck(handle);
46     printf("\n");
47
48
49     /* ***** *
50     *           *
51     *   Initialize Freq.Block   *
52     *           *
53     * ***** */
54
55     float startFrequency    = 100;        //100Hz
56     float stopFrequency     = 100e3;      //100kHz
57     float frequencyCount    = 80;
58     float precision         = 1;
59     float amplitude         = 0.1;        //V
60     byte  scale             = 1;          //log
61     printf("Set Setup-config: Frequency-block (%fHz .. %fHz, scale=%i,
prec=%f, amplitude=%fV)\n", startFrequency, stopFrequency, scale,
precision, amplitude);
62     numberOfBytes = 0x16 + 3;
63     cmd = (byte*)malloc(sizeof(byte)*numberOfBytes);
64     cmd[0] = 0xB6; cmd[1] = numberOfBytes-3; cmd[2] = 0x03;
65
66     unsigned long tmp = *(unsigned long*)&startFrequency;
67     byte counter = 3;

```

```

68         cmd[counter++] = (tmp>>24)&0xFF;
69         cmd[counter++] = (tmp>>16)&0xFF;
70         cmd[counter++] = (tmp>>8)&0xFF;
71         cmd[counter++] = (tmp)&0xFF;
72
73         tmp = *(unsigned long*)&stopFrequency;
74         cmd[counter++] = (tmp>>24)&0xFF;
75         cmd[counter++] = (tmp>>16)&0xFF;
76         cmd[counter++] = (tmp>>8)&0xFF;
77         cmd[counter++] = (tmp)&0xFF;
78
79         tmp = *(unsigned long*)&frequencyCount;
80         cmd[counter++] = (tmp>>24)&0xFF;
81         cmd[counter++] = (tmp>>16)&0xFF;
82         cmd[counter++] = (tmp>>8)&0xFF;
83         cmd[counter++] = (tmp)&0xFF;
84
85         cmd[counter++] = scale;
86
87         tmp = *(unsigned long*)&precision;
88         cmd[counter++] = (tmp>>24)&0xFF;
89         cmd[counter++] = (tmp>>16)&0xFF;
90         cmd[counter++] = (tmp>>8)&0xFF;
91         cmd[counter++] = (tmp)&0xFF;
92
93         tmp = *(unsigned long*)&litide;
94         cmd[counter++] = (tmp>>24)&0xFF;
95         cmd[counter++] = (tmp>>16)&0xFF;
96         cmd[counter++] = (tmp>>8)&0xFF;
97         cmd[counter++] = (tmp)&0xFF;
98
99         cmd[numberOfBytes-1] = 0xB6;
100        writeToDevice(handle, cmd, numberOfBytes);
101        free(cmd);
102
103        readAck(handle);
104        printf("\n");
105
106
107        /* ***** *
108        *                               *
109        *      Set FrontEnd Settings    *
110        *                               *
111        * ***** */
112        printf("Clear Channel Stack\n");
113        numberOfBytes = 6;
114        cmd = (byte*)malloc(sizeof(byte)*numberOfBytes);
115        cmd[0] = 0xB0;    cmd[1] = 0x03;    cmd[numberOfBytes-1] = cmd[0];
116        cmd[2] = 0xFF;    cmd[3] = 0xFF;    cmd[4] = 0xFF; //Delete channel
stack
117        writeToDevice(handle, cmd, numberOfBytes);
118        readAck(handle);
119
120        printf("\n");    printf("Set Channel Stack: to channel 1\n");

```

```

121     cmd[3] = 0x02;    //mode = 4Pt measurement
122     cmd[4] = 0x01;    //add channel: Channel 1 (BNC),
123     cmd[5] = 0x01;    //1000hm Range,
124     writeToDevice(handle, cmd, numberOfBytes);
125     readAck(handle);
126     printf("\n");
127
128     free(cmd);
129
130     /* ***** *
131     *                               *
132     *      Start Measurement      *
133     *                               *
134     * ***** */
135     byte numberOfSpecs = 1;
136     printf("Start Measurement\n");
137     numberOfBytes = 6;
138     cmd = (byte*)malloc(sizeof(byte)*numberOfBytes);
139     cmd[0] = 0xB8;  cmd[1] = 0x03;
140     cmd[2] = 0x01;
141     cmd[3] = 0;
142     cmd[4] = 0;
143     cmd[numberOfBytes-1] = 0xB8;
144     writeToDevice(handle, cmd, numberOfBytes);
145     free(cmd);
146     readAck(handle);
147     printf("\n");
148
149
150     /* ***** *
151     *                               *
152     *      Receive Specs           *
153     *                               *
154     * ***** */
155     readBuffer = malloc(14);    //3Byte Framing, 2Byte idNumber, 4Byte RE,
4Byte Im
156     byte j;
157     byte i;
158     UINT8 ch;
159     UINT16 id;
160     UINT32 tmp32;
161     float re, im;
162     for(j=0; j<numberOfSpecs; j++){
163         printf("Spec%i:\n", j+1);
164         printf("ch\tid\tre\tim\n");
165         for(i=0; i<frequencyCount; i++){
166
167             readData(handle, readBuffer, 14);
168
169             ch = readBuffer[2];
170             id = (readBuffer[3]<<8) + readBuffer[4];
171             tmp32 = (readBuffer[5]<<24) + (readBuffer[6]<<16) +
(readBuffer[7]<<8) + (readBuffer[8]);
172             re = *(float*)&tmp32;

```

```

173         tmp32 = (readBuffer[9]<<24) + (readBuffer[10]<<16) +
174         (readBuffer[11]<<8) + (readBuffer[12]);
175         im = *(float*)&tmp32;
176         printf("%i\t%i\t%f\t%f\n", ch, id, re, im);
177     }
178     printf("\n");
179 }
180
181 printf("Stop Measurement");
182 numberOfBytes = 6;
183 cmd = (byte*)malloc(sizeof(byte)*numberOfBytes);
184 cmd[0] = 0xB8; cmd[1] = 0x03;
185 cmd[2] = 0x00;
186 cmd[3] = 0;
187 cmd[4] = 0;
188 cmd[numberOfBytes-1] = 0xB8;
189 writeToDevice(handle, cmd, numberOfBytes);
190 free(cmd);
191 readAck(handle);
192 printf("\n");
193
194 CloseHandle(handle); //Closing the Serial Port
195
196 // sprintf();
197
198 return 1;
199 }
200
201 /*
202  * Connect to COM6 if available
203  * @param handle - the pointer to USB-Conection-Handle
204  * @return True if connection was successfull else false
205  */
206 byte connectToDevice(HANDLE* hSerial){
207     *hSerial = CreateFile( "\\\\.\\COM6",
208                           GENERIC_READ|GENERIC_WRITE,
209                           0,
210                           NULL,
211                           OPEN_EXISTING,
212                           FILE_ATTRIBUTE_NORMAL,
213                           NULL );
214     if (hSerial == INVALID_HANDLE_VALUE)
215     {
216         fprintf(stderr, "Error\n");
217         return 0;
218     }
219
220     return 1;
221 }
222
223 void writeToDevice(HANDLE handle, byte* cmd, DWORD dataCount){
224
225     byte i;

```

```

226     for(i=0; i<dataCount;i++)
227         printf("%.2X ", cmd[i]);
228     printf("\n");
229
230     WriteFile(handle, cmd, dataCount, NULL, NULL);
231
232
233 }
234
235 /** Tries to Read ACK Frame
236  *
237  * @return true if ACK is received, else false
238  */
239 byte readAck(HANDLE handle){
240
241     int i = 0;
242     byte* readBuffer;
243     readBuffer = (byte*)malloc(sizeof(byte)*4);
244
245     readData(handle, readBuffer, 4);
246
247     printf("ACK-Frame: ");
248     for(i=0; i<4;i++){
249         printf("%.2X ", readBuffer[i]);
250     }
251     printf("\n");
252
253     if(readBuffer[2] == 0x83){
254         free(readBuffer);
255         return TRUE;    //ACK
256     }else{
257         free(readBuffer);
258         return FALSE;   //NOT ACK
259     }
260 }
261
262 void readData(HANDLE handle, byte* buffer, int bytesToRead){
263     int i;
264     for (i = 0; i < bytesToRead; i++) {
265         unsigned char ch = 0;
266         DWORD read = 0;
267
268         /* Wait for characters */
269         while(!read) {
270             ReadFile(handle, &ch, 1, &read, NULL);
271             if (read) {
272                 buffer[i] = ch;
273             }
274         }
275     }
276 }

```